Smart Pay App

A Robust Financial Transaction System

Presented by :
EMAN MAQSOOD (030)
Mahnoor saddique(110)
Alishba Arshad (139)
Software Construction and Development

Introduction to Smart Pay

The Smart Pay App is a sophisticated, desktop-based financial transaction management system designed to simulate a personal banking environment. Our goal was to create a secure and intuitive application, demonstrating the practical application of core software engineering principles.

Desktop-Based

Robust and reliable system.

Personal Banking

Simulated environment.

Engineering Principles

MVC, Agile, Testing applied.

Problem & Scope

The need for secure, localized personal finance management and simulated digital payments drove the development of Smart Pay. Our scope focused on key functionalities to address this need.

Problem Statement

Current solutions often lack integrated security and localized features for personal finance. There is a clear demand for a secure, user-friendly platform that allows individuals to track expenses, manage income, and simulate digital payment processes within a controlled environment.

Defined Scope

Financial Management

Income and expense tracking.

P2P Digital Payments

Simulated peer-to-peer transfers.

Visual Reporting

Receipt generation and history.

Key Features Overview

Smart Pay delivers a comprehensive suite of features designed for a seamless user experience, from secure access to detailed financial insights.

User Authentication

Robust login and registration ensure secure access.

Interactive Dashboard

Real-time balance, visual QR code generation for payments.

Transaction Management

Effortless P2P money transfers and income/expense capitalization.

Reporting & History

Searchable transaction history and downloadable PDF receipts.

Technology Stack

Our application is built on a modern and efficient technology stack, ensuring performance, scalability, and maintainability.

1

Python 3.x

Core programming language.

2

Tkinter

Custom-styled GUI Framework for a modern look.

3

SQL Server / SQLite

Robust database solutions for production and development.

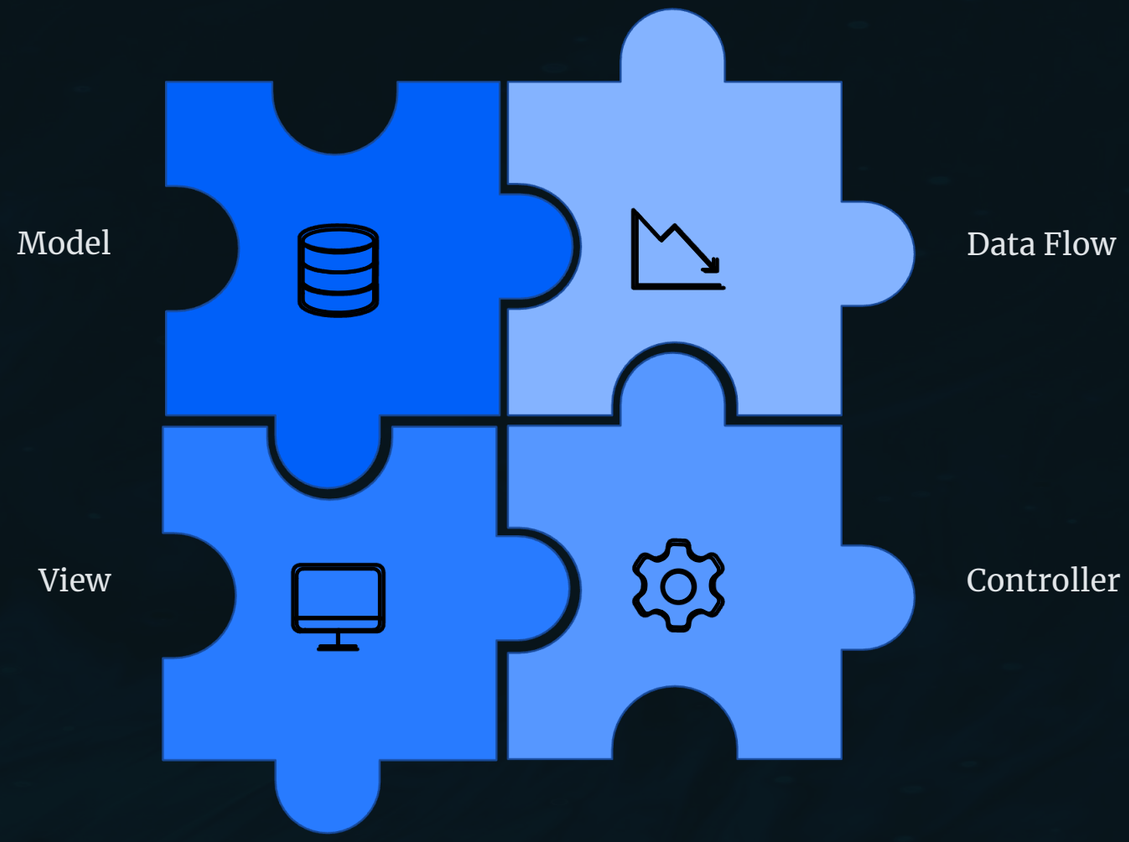
4

Key Libraries

pyodbc, qrcode, fpdf, unittest for enhanced functionality.

System Architecture: MVC

The Smart Pay App adheres to the Model-View-Controller (MVC) architectural pattern, fostering modularity, testability, and easier maintenance.



This decoupling of concerns allows independent development and updates without affecting other components.

Agile Development Process

We adopted an Agile methodology, embracing iterative and incremental development to adapt to evolving requirements and mitigate risks effectively.

1

Iterative & Incremental

Flexible development cycles for continuous improvement.

2

Evolving Requirements

Ability to integrate new features like Account # support seamlessly.

3

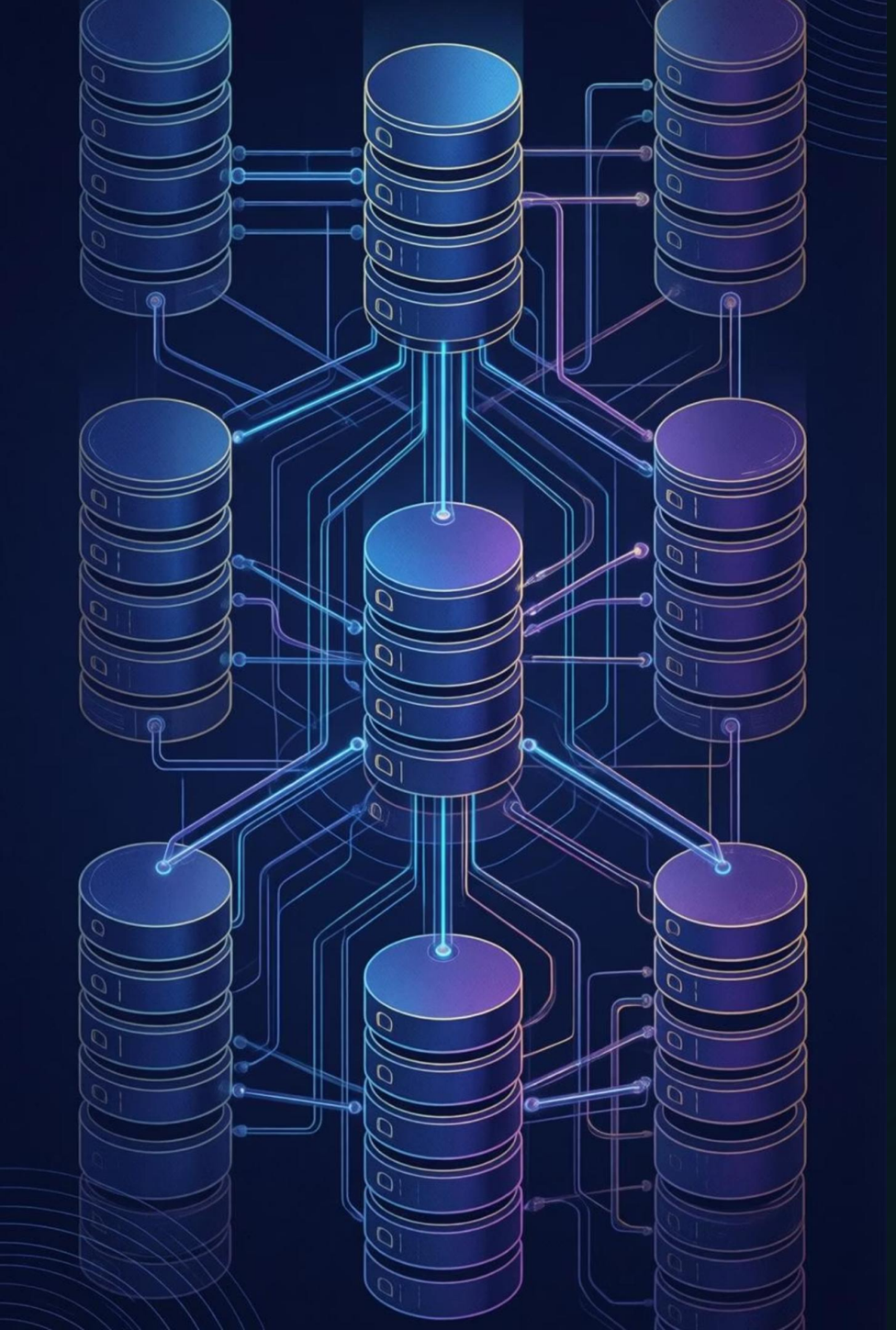
Risk Management

Prioritized testing of critical paths, e.g., Database connectivity.

4

Phased Delivery

Sequential feature rollout: Authentication, Dashboard, then Transfers.



Database Design & Integrity

Our database is meticulously designed to ensure data integrity, security, and efficient management of financial information.

- ## Users Table

  Stores secure credentials and current balance for each user.

- ## Transactions Table

  Records details of all financial movements, linking senders and receivers.

- ## Categories

  Provides taxonomies (e.g., Food, Salary) for organized reporting.

Data Integrity & Security

Foreign Keys are extensively used to maintain relationships between tables, ensuring consistent and reliable data. This design minimizes anomalies and enhances overall database security.

Advanced Engineering Practices

We implemented rigorous engineering practices to build a robust, maintainable, and high-quality software product.

Version Control

Git with strict branching strategy (Main, Develop, Feature) ensured code integrity and collaborative development.

Comprehensive Testing

Unit Tests using Python's unittest framework, addressing edge cases like negative amounts and invalid user inputs.

Continuous Refactoring

Proactive code improvements, such as migrating from "Button Logic" to "Controller Logic," reduced technical debt and improved modularity.

Security & Future Vision

Beyond current implementation, we have clear plans for enhancing security and expanding the app's capabilities.

Robust Security Implementation

- **Data Protection:** Parameterized Queries safeguard against SQL Injection attacks, securing user data.
- **Input Validation:** Strict type checking before database calls prevents malformed data entries.
- **Error Handling:** Graceful degradation with generic error messages protects users from technical details and hides internal system information.

Future Enhancements Roadmap

→ Enhanced Security

Implement bcrypt for robust password hashing.

→ Cloud Integration

Migrate DB to Azure/AWS for universal access and scalability.

→ Mobile Port

Convert core logic to Kivy/Flutter for cross-platform mobile applications.

→ Real Banking APIs

Integrate APIs like Plaid for real-world transaction data.